

```
134
135     for mark in s[2]:
136         if mark > 80:
137             above = above + 1
138
139     if above >= 2:
140         count = count + 1
141
142 print()
143 print("Above 80 in at least two subjects:")
144 print(count)
145
146 highest_s3 = data[0]
147
148 for s in data:
149     if s[2][2] > highest_s3[2][2]:
150         highest_s3 = s
151
152 print()
153 print("Highest in Subject 3:")
154 print(highest_s3[1], "->", highest_s3[2][2])
155
156 print()
157 print("Data Structure:")
158 print("List is used for student records.")
159 print("Tuple is used for each student record.")
160 print("List is used for marks.")
161 print("Student ID is used to identify each student.")
```



Run



Save

```
101 ("Top 2:")
102 (sorted_data[0][1], "-> %.2f" % sorted_data[0][4])
103 (sorted_data[1][1], "-> %.2f" % sorted_data[1][4])
104
105 ()
106 ("Subject-wise Highest:")
107
108 in range(3):
109 igh = students[0][2][i]
110
111 or s in students:
112     if s[2][i] > high:
113         high = s[2][i]
114
115 rint("Subject", i + 1, "->", high)
116
117 ()
118 ("Subject-wise Average:")
119
120 in range(3):
121 otal = 0
122
123 or s in students:
124     total = total + s[2][i]
125
126 verage = total / len(students)
127
128 rint("Subject", i + 1, "-> %.2f" % average)
129
130 = 0
131
132 in data:
133 bove = 0
```



```
69 ()
70 ("Average > 75:")
71
72 in data:
73 f s[4] > 75:
74     print(s[1])
75
76 ()
77 ("Duplicate Name:")
78
79 = []
80
81 in data:
82 f s[1] in names:
83     print(s[1])
84 lse:
85     names.append(s[1])
86
87 d_data = data[:]
88
89 in range(len(sorted_data)):
90 or j in range(i + 1, len(sorted_data)):
91     if sorted_data[i][4] < sorted_data[j][4]:
92         sorted_data[i], sorted_data[j] = sorted_data[j], sorted_data[i]
93
94 ()
95 ("Sorted Based on Average:")
96
97 in sorted_data:
98 rint(s[1], "-> %.2f" % s[4])
99
100 ()
101 ("Top 2:")
```

```
34 in data:
35 rint(s[1], "-> %.2f" % s[4])
36
37 ()
38 ("Passed All Subjects:")
39
40 in data:
41 f min(s[2]) >= 50:
42     print(s[1])
43
44 ()
45 ("Failed At Least One:")
46
47 in data:
48 f min(s[2]) < 50:
49     print(s[1])
50
51 st = data[0]
52 t = data[0]
53
54 in data:
55 f s[3] > highest[3]:
56     highest = s
57
58 f s[3] < lowest[3]:
59     lowest = s
60
61 ()
62 ("Highest Total:")
63 (highest[1], "->", highest[3])
64
65 ()
66 ("Lowest Total:")
```

Your Code

```
1 nts = [  
2 " A001 ", "ARUN KUMAR", [78, 85, 92]),  
3 "A002", "Bala", [45, 67, 52]),  
4 " a003", "arun kumar", [88, 91, 95]),  
5 "A004", "CHARAN", [72, 76, 81]),  
6 "A005", "Divya ", [55, 48, 62])  
7  
8  
9 = []  
10  
11 in students:  
12 id = s[0].strip().upper()  
13 ame = s[1].strip().lower()  
14 arks = s[2]  
15 otal = sum(marks)  
16 vg = total / 3  
17 ata.append([sid, name, marks, total, avg])  
18  
19 ("Normalized Records:")  
20 ()  
21  
22 in data:  
23 rint(s[0], s[1], s[2])  
24  
25 ()  
26 ("Totals:")  
27  
28 in data:  
29 rint(s[1], "->", s[3])  
30  
31 ()  
32 ("Averages:")
```

Input

stdin input

Output

```
Normalized Records:  
  
A001 arun kumar [78, 85, 92]  
A002 bala [45, 67, 52]  
A003 arun kumar [88, 91, 95]  
A004 charan [72, 76, 81]
```

Failed At Least One:

bala

divya

Highest Total:

arun kumar → 274

Lowest Total:

bala → 164

Average > 75:

arun kumar

arun kumar

charan

Duplicate Name:

arun kumar

Top 2:

arun kumar → 91.33

arun kumar → 85.00

Subject-wise Highest:

Subject 1 → 88

Subject 2 → 91

Subject 3 → 95

Subject-wise Average:

Subject 1 → 67.60

Subject 2 → 73.40

Subject 3 → 78.40

Q8

Code Analysis and Debugging

— List + Tuple + String

10 marks

**Q9**

Word Frequency and Data

Analysis — String + List + Tuple

10 marks

**Q10**

Integrated Student Data

Analysis — String + List + Tuple

10 marks



10/10 answered

15. Identify the student who has the highest mark in Subject 3.

16. Explain which data structure is appropriate for student records, marks, and individual student identity, and justify your answer.

Expected Output:

Normalized Records:

A001 arun kumar [78, 85, 92]

A002 bala [45, 67, 52]

A003 arun kumar [88, 91, 95]

A004 charan [72, 76, 81]

A005 divya [55, 48, 62]

Totals:

arun kumar → 255

bala → 164

arun kumar → 274

charan → 229

divya → 165

Averages:

arun kumar → 85.00

bala → 54.67

arun kumar → 91.33

charan → 76.33

divya → 55.00

Passed All Subjects:

arun kumar

arun kumar

charan

Failed At Least One:

QUESTIONS

Q1

 Student Performance Analysis — List + Tuple
10 marks

Q2

 Customer Name Analysis — String + List
10 marks

Q3

 Product Inventory Analysis — List + Tuple
10 marks

Q4

 Text Analysis — String + List
10 marks

Q5

 Employee Skill Analysis — Tuple + List + String
10 marks

Q6

 Transaction Analysis — Tuple + List
10 marks

Q7

 Password Security Analysis — String + List
10 marks

Q10 Integrated Student Data Analysis — String + List + Tuple

10 marks

A college maintains:

```
students = [
    (" A001 ", "ARUN KUMAR", [78, 85, 92]),
    ("A002", "Bala", [45, 67, 52]),
    (" a003", "arun kumar", [88, 91, 95]),
    ("A004", "CHARAN", [72, 76, 81]),
    ("A005", "Divya ", [55, 48, 62])
]
```

Each record contains:

(Student ID, Student Name, Marks)

Analyze the complete dataset:

1. Normalize all student IDs using strip() and upper().
2. Normalize all student names using strip() and lower().
3. Calculate total and average marks.
4. Identify students who passed all subjects.
5. Identify students who failed at least one subject.
6. Find the highest total.
7. Find the lowest total.
8. Identify students with average above 75.
9. Detect duplicate student names after normalization.
10. Sort students based on average marks.
11. Find the top two students.
12. Find the subject-wise highest marks.
13. Find the subject-wise average marks.
14. Count the number of students who scored above 80 in at least two subjects.
15. Identify the student who has the highest mark in Subject 3.



```
28 print("Most Frequent Word:", most)
29
30 t = []
31 for w in unique:
32     t.append((w, words.count(w)))
33
34 print("Tuples:")
35 print(t)
36
37 print("Words having frequency greater than 1:")
38 for w in unique:
39     if words.count(w) > 1:
40         print(w)
41
42 longest = unique[0]
43 for w in unique:
44     if len(w) > len(longest):
45         longest = w
46
47 print("Longest Word:", longest)
48
49 print("Words beginning with d:")
50 for w in unique:
51     if w[0] == "d":
52         print(w)
```



Run



Save

10/10 answered

Your Code

```
1 text = "data science uses data analysis and data visualization"
2
3 words = text.split()
4
5 unique = []
6 for w in words:
7     if w not in unique:
8         unique.append(w)
9
10 print("Total Words: 9")
11 print("Unique Words:")
12 print(unique)
13
14 print("Frequency:")
15 for w in unique:
16     print(w, "->", words.count(w))
17
18 print("Repeated Words:")
19 for w in unique:
20     if words.count(w) > 1:
21         print(w)
22
23 most = unique[0]
24 for w in unique:
25     if words.count(w) > words.count(most):
26         most = w
27
28 print("Most Frequent Word:", most)
29
30 t = []
31 for w in unique:
```

Input

stdin input

Output

```
Total Words: 9
Unique Words:
['data', 'science', 'uses',
'analysis', 'and',
'visualization']
Frequency:
```

Q7Password Security Analysis —
String + List
10 marks**Q8**Code Analysis and Debugging
— List + Tuple + String
10 marks**Q9**Word Frequency and Data
Analysis — String + List + Tuple
10 marks**Q10**Integrated Student Data
Analysis — String + List + Tuple
10 marks

10/10 answered

Frequency:

data → 3

science → 1

uses → 1

analysis → 1

and → 1

visualization → 1

Most Frequent:

data

Repeated Words:

data

Longest Word:

visualization

Your Code

```
1 text = "data science uses data analysis and data visualization"
2
3 words = text.split()
4
5 unique = []
6 for w in words:
7     if w not in unique:
8         unique.append(w)
9
10 print("Total Words: 9")
11 print("Unique Words:")
12 print(unique)
```

Input

stdin input

Output

```
Total Words: 9
Unique Words:
['data', 'science', 'uses',
'analysis', 'and',
'visualization']
Frequency:
```

QUESTIONS

Q1

Student Performance Analysis — List + Tuple
10 marks



Q2

Customer Name Analysis — String + List
10 marks



Q3

Product Inventory Analysis — List + Tuple
10 marks



Q4

Text Analysis — String + List
10 marks



Q5

Employee Skill Analysis — Tuple + List + String
10 marks



Q6

Transaction Analysis — Tuple + List
10 marks



Q7



Q9 Word Frequency and Data Analysis — String + List + Tuple

10 marks

Consider:

text = "data science uses data analysis and data visualization"

Analyze the string using Python.

Tasks

1. Convert the sentence into a list of words.
2. Count the total words.
3. Find unique words.
4. Find repeated words.
5. Count the frequency of every word.
6. Identify the most frequently occurring word.
7. Create tuples of the form:
(word, frequency)
8. Sort the tuples based on frequency.
9. Find words having frequency greater than 1.
10. Identify the longest word.
11. Find words beginning with 'd'.
12. Calculate the percentage of repeated words.

Expected Output:

Total Words: 9

Unique Words:

['data', 'science', 'uses', 'analysis', 'and', 'visualization']

Frequency:

Q8Code Analysis and Debugging
— List + Tuple + String
10 marks**Q9**Word Frequency and Data
Analysis — String + List + Tuple
10 marks**Q10**Integrated Student Data
Analysis — String + List + Tuple
10 marks

10/10 answered

9. Analyze what happens if record[0] is changed instead.
10. Explain why modifying record[0] directly is not allowed.
Expected Output:
['arun', 'bala', 'charan']

Your Code

```
1 records = [  
2     ("Arun ", "Python"),  
3     ("ARUN", "Python"),  
4     ("Bala ", "Java"),  
5     ("bala", "Java"),  
6     ("Charan", "Python")  
7 ]  
8  
9 names = []  
10  
11 for record in records:  
12     name = record[0].strip().lower()  
13  
14     if name not in names:  
15         names.append(name)  
16  
17 print(names)
```



Run



Save

Input

stdin input

Output

['arun', 'bala', 'charan']

QUESTIONS

Q1Student Performance Analysis
— List + Tuple
10 marks**Q2**Customer Name Analysis —
String + List
10 marks**Q3**Product Inventory Analysis —
List + Tuple
10 marks**Q4**Text Analysis — String + List
10 marks**Q5**Employee Skill Analysis — Tuple
+ List + String
10 marks**Q6**Transaction Analysis — Tuple +
List
10 marks**Q7**Password Security Analysis —
String + List
10 marks**Q8 Code Analysis and Debugging — List + Tuple + String**

10 marks

Consider the following program:

```
records = [  
    ("Arun ", "Python"),  
    ("ARUN", "Python"),  
    ("Bala ", "Java"),  
    ("bala", "Java"),  
    ("Charan", "Python")  
]  
  
names = []  
  
for record in records:  
    name = record[0].strip().lower()  
  
    if name not in names:  
        names.append(name)  
  
print(names)
```

Analyze the program:

1. What happens during each iteration?
2. What value is stored in record?
3. What does record[0] represent?
4. Why are strip() and lower() used together?
5. Why does "Arun" appear only once?
6. Why does "Bala" appear only once?
7. What is the final output?
8. Modify the logic to find unique employees and their skills.
9. Analyze what happens if record[0] is changed instead.


```
32
33 print("Valid:")
34 for p in valid:
35     print(p)
36
37 print("\nInvalid:")
38 for p, failed in invalid:
39     print(p, "-", ", ".join(failed))
40
41 print("\nValid count:", len(valid))
42 print("Invalid count:", len(invalid))
43
44 m = max(len(p) for p in passwords)
45 print("Longest password:")
46 for p in passwords:
47     if len(p) == m:
48         print(p)
49
50 print("\nPasswords containing python:")
51 for p in passwords:
52     if "python" in p.lower():
53         print(p)
54
55 print("\nPasswords sorted by length:")
56 for p in sorted(passwords, key=len):
57     print(p)
58
59 percentage = (len(valid) / len(passwords)) * 100
60 print("\nValid percentage: %.2f%%" % percentage)
```



Run



Save

Your Code

```
1 passwords = [  
2     "Python@123",  
3     "python123",  
4     "PYTHON@123",  
5     "Py@45",  
6     "Python123",  
7     "Java#4567",  
8     "Admin@2026"  
9 ]  
10  
11 valid = []  
12 invalid = []  
13  
14 for p in passwords:  
15     failed = []  
16  
17     if len(p) < 8:  
18         failed.append("At least 8 characters")  
19     if not any(c.isupper() for c in p):  
20         failed.append("Uppercase")  
21     if not any(c.islower() for c in p):  
22         failed.append("Lowercase")  
23     if not any(c.isdigit() for c in p):  
24         failed.append("Digit")  
25     if not any(not c.isalnum() for c in p):  
26         failed.append("Special character")  
27  
28     if len(failed) == 0:  
29         valid.append(p)  
30     else:  
31         invalid.append((p, failed))  
32
```

Input

stdin input

Output

Valid:
Python@123
Java#4567
Admin@2026
Invalid:

Code Analysis and Debugging
— List + Tuple + String
10 marks

Q9 ✓
Word Frequency and Data
Analysis — String + List + Tuple
10 marks

Q10 ✓
Integrated Student Data
Analysis — String + List + Tuple
10 marks

10/10 answered

Expected Output:

Valid:

Python@123

Java#4567

Admin@2026

Invalid:

python123 → No uppercase, No special character

PYTHON@123 → No lowercase

Py@45 → Less than 8 characters

Python123 → No special character

Valid Count: 3

Invalid Count: 4

Longest Password:

Python@123

Python-related passwords:

Python@123

python123

PYTHON@123

Python123

Valid Percentage: 42.86%

Your Code

```
1 passwords = [  
2     "Python@123",  
3     "python123",  
4     "PYTHON@123",
```

Input

stdin input

QUESTIONS

Q1Student Performance Analysis — List + Tuple
10 marks**Q2**Customer Name Analysis — String + List
10 marks**Q3**Product Inventory Analysis — List + Tuple
10 marks**Q4**Text Analysis — String + List
10 marks**Q5**Employee Skill Analysis — Tuple + List + String
10 marks**Q6**Transaction Analysis — Tuple + List
10 marks**Q7**Password Security Analysis — String + List
10 marks**Q7 Password Security Analysis — String + List**

10 marks

A system receives:

```
passwords = [  
    "Python@123",  
    "python123",  
    "PYTHON@123",  
    "Py@45",  
    "Python123",  
    "Java#4567",  
    "Admin@2026"  
]
```

A valid password must have:

- At least 8 characters
- Uppercase
- Lowercase
- Digit
- Special character

Analyze:

1. Check every password against all five conditions.
2. Identify valid passwords.
3. Identify invalid passwords.
4. For each invalid password, identify the failed conditions.
5. Count valid and invalid passwords.
6. Find the longest password.
7. Find passwords containing "python" irrespective of case.
8. Sort passwords based on length.
9. Determine the percentage of valid passwords.

```
21 print("Largest Withdrawal:", largest_withdraw)
22
23 print("\nTransactions greater than 4000:")
24 for x in transactions:
25     if x[2] > 4000:
26         print(x)
27
28 dcount = sum(1 for x in transactions if x[1] == "Deposit")
29 wcount = sum(1 for x in transactions if x[1] == "Withdraw")
30
31 print("\nDeposit Count:", dcount)
32 print("Withdrawal Count:", wcount)
33
34 average = sum(x[2] for x in transactions) / len(transactions)
35 print("Average Transaction:", average)
36
37 print("\nSorted Transactions:")
38 for x in sorted(transactions, key=lambda x: x[2]):
39     print(x)
40
41 print("\nWithdrawals exceed 50% of deposits:")
42 if withdraw > deposit * 0.5:
43     print("Yes")
44 else:
45     print("No")
```



Run



Save

Your Code

```
1 transactions = [  
2     ("T101", "Deposit", 5000),  
3     ("T102", "Withdraw", 2000),  
4     ("T103", "Deposit", 8000),  
5     ("T104", "Withdraw", 1500),  
6     ("T105", "Deposit", 3000),  
7     ("T106", "Withdraw", 7000)  
8 ]  
9  
10 deposit = sum(x[2] for x in transactions if x[1] == "Deposit")  
11 withdraw = sum(x[2] for x in transactions if x[1] == "Withdraw")  
12  
13 print("Total Deposit:", deposit)  
14 print("Total Withdrawal:", withdraw)  
15 print("Net Balance:", deposit - withdraw)  
16  
17 largest_deposit = max(x[2] for x in transactions if x[1] == "Deposit")  
18 largest_withdraw = max(x[2] for x in transactions if x[1] == "Withdraw")  
19  
20 print("\nLargest Deposit:", largest_deposit)  
21 print("Largest Withdrawal:", largest_withdraw)  
22  
23 print("\nTransactions greater than 4000:")  
24 for x in transactions:  
25     if x[2] > 4000:  
26         print(x)  
27  
28 dcount = sum(1 for x in transactions if x[1] == "Deposit")  
29 wcount = sum(1 for x in transactions if x[1] == "Withdraw")  
30  
31 print("\nDeposit Count:", dcount)  
32 print("Withdrawal Count:", wcount)
```

Input

stdin input

Output

```
Total Deposit: 16000  
Total Withdrawal: 10500  
Net Balance: 5500  
  
Largest Deposit: 8000  
Largest Withdrawal: 7000
```


Q7Password Security Analysis —
String + List
10 marks**Q8**Code Analysis and Debugging
— List + Tuple + String
10 marks**Q9**Word Frequency and Data
Analysis — String + List + Tuple
10 marks**Q10**Integrated Student Data
Analysis — String + List + Tuple
10 marks

10/10 answered

Expected Output:

Total Deposit: 16000

Total Withdrawal: 10500

Net Balance: 5500

Largest Deposit: 8000

Largest Withdrawal: 7000

Transactions > 4000:

T101

T103

T106

Deposits: 3

Withdrawals: 3

Average Transaction: 4416.67

Withdrawals > 50% of Deposits:

Yes

Your Code

```
1 transactions = [  
2     ("T101", "Deposit", 5000),  
3     ("T102", "Withdraw", 2000),  
4     ("T103", "Deposit", 8000),  
5     ("T104", "Withdraw", 1500),  
6     ("T105", "Deposit", 3000),  
7     ("T106", "Withdraw", 7000)  
8 ]  
9  
10 deposit = sum(x[2] for x in transactions if x[1] == "Deposit")
```

Input

stdin input

Output

```
Total Deposit: 16000  
Total Withdrawal: 10500  
Net Balance: 5500
```


CO4_AT2_CASE STUDY[← Back](#)

QUESTIONS

Q1

Student Performance Analysis
— List + Tuple
10 marks

Q2

Customer Name Analysis —
String + List
10 marks

Q3

Product Inventory Analysis —
List + Tuple
10 marks

Q4

Text Analysis — String + List
10 marks

Q5

Employee Skill Analysis — Tuple
+ List + String
10 marks

Q6

Transaction Analysis — Tuple +
List
10 marks

Q7**Q6 Transaction Analysis — Tuple + List**

10 marks

A bank stores transactions:

```
transactions = [  
    ("T101", "Deposit", 5000),  
    ("T102", "Withdraw", 2000),  
    ("T103", "Deposit", 8000),  
    ("T104", "Withdraw", 1500),  
    ("T105", "Deposit", 3000),  
    ("T106", "Withdraw", 7000)  
]
```

Analyze the transaction history.

Tasks

1. Calculate total deposits.
2. Calculate total withdrawals.
3. Calculate the net balance.
4. Find the largest deposit.
5. Find the largest withdrawal.
6. Identify all transactions greater than ₹4,000.
7. Count deposits and withdrawals.
8. Find the average transaction amount.
9. Sort transactions based on amount.
10. Identify whether total withdrawals exceed 50% of total deposits.

Expected Output:

Total Deposit: 16000

```
24
25 skill_counts = {}
26 for skill in all_skills:
27     skill_counts[skill] = skill_counts.get(skill, 0) + 1
28
29 print("\nSkill Count:")
30 for skill in sorted(skill_counts):
31     print(skill + " :", skill_counts[skill])
32
33 most_common = max(skill_counts, key=skill_counts.get)
34 print("\nMost Common Skill:", most_common)
35
36 print("\nEmployees with 3 skills:")
37 print(", ".join(name for eid, name, skills in employees if len(skills)
38
39 print("\nWithout Python:")
40 print(", ".join(name for eid, name, skills in employees if "Python" not
41
42 sorted_emp = sorted(employees, key=lambda x: len(x[2]), reverse=True)
43 print("\nSorted by number of skills:")
44 for eid, name, skills in sorted_emp:
45     print(name, ":", len(skills))
46
47 print("\nSuitable for Python + ML:")
48 print(", ".join(name for eid, name, skills in employees if "Python" in
```



Run



Save

Your Code

```
1 employees = [  
2     ("E101", "Arun", ["Python", "SQL", "ML"]),  
3     ("E102", "Bala", ["Java", "SQL", "HTML"]),  
4     ("E103", "Charan", ["Python", "ML", "SQL"]),  
5     ("E104", "Divya", ["Python", "Java", "Cloud"]),  
6     ("E105", "Esha", ["Python", "ML", "Cloud"])  
7 ]  
8  
9 print("Python:")  
10 print(", ".join(name for eid, name, skills in employees if "Python" in skills))  
11  
12 print("\nPython + ML:")  
13 print(", ".join(name for eid, name, skills in employees if "Python" in skills and "ML" in skills))  
14  
15 print("\nSQL but not Java:")  
16 print(", ".join(name for eid, name, skills in employees if "SQL" in skills and "Java" not in skills))  
17  
18 print("\nJava or Cloud:")  
19 print(", ".join(name for eid, name, skills in employees if "Java" in skills or "Cloud" in skills))  
20  
21 all_skills = []  
22 for eid, name, skills in employees:  
23     all_skills.extend(skills)  
24  
25 skill_counts = {}  
26 for skill in all_skills:  
27     skill_counts[skill] = skill_counts.get(skill, 0) + 1  
28  
29 print("\nSkill Count:")  
30 for skill in sorted(skill_counts):  
31     print(skill + " :", skill_counts[skill])
```

Input

Output

```
Python:  
Arun, Charan, Divya, Esha  
  
Python + ML:  
Arun, Charan, Esha
```

10 marks

Q7Password Security Analysis —
String + List
10 marks**Q8**Code Analysis and Debugging
— List + Tuple + String
10 marks**Q9**Word Frequency and Data
Analysis — String + List + Tuple
10 marks**Q10**Integrated Student Data
Analysis — String + List + Tuple
10 marks

10/10 answered

Expected Output:

Python:

Arun, Charan, Divya, Esha

Python + ML:

Arun, Charan, Esha

SQL but not Java:

Arun, Charan

Java or Cloud:

Bala, Divya, Esha

Employees with 3 skills:

Arun, Bala, Charan, Divya, Esha

Without Python:

Bala

Most Common Skill:

Python

Suitable for Python + ML:

Arun, Charan, Esha

Your Code

```
1 employees = [  
2     ("E101", "Arun", ["Python", "SQL", "ML"]),  
3     ("E102", "Bala", ["Java", "SQL", "HTML"]),  
4     ("E103", "Charan", ["Python", "ML", "SQL"]),  
5     ("E104", "Divya", ["Python", "Java", "Cloud"]),  
6     ("E105", "Esha", ["Python", "ML", "Cloud"])]
```

Input

stdin input

Output

CO4_AT2_CASE STUDY[← Back](#)

QUESTIONS

Q1Student Performance Analysis
— List + Tuple
10 marks**Q2**Customer Name Analysis —
String + List
10 marks**Q3**Product Inventory Analysis —
List + Tuple
10 marks**Q4**Text Analysis — String + List
10 marks**Q5**Employee Skill Analysis — Tuple
+ List + String
10 marks**Q6**Transaction Analysis — Tuple +
List
10 marks**Q7****Q5 Employee Skill Analysis — Tuple + List + String**

10 marks

An organization stores:

employees = [

("E101", "Arun", ["Python", "SQL", "ML"]),

("E102", "Bala", ["Java", "SQL", "HTML"]),

("E103", "Charan", ["Python", "ML", "SQL"]),

("E104", "Divya", ["Python", "Java", "Cloud"]),

("E105", "Esha", ["Python", "ML", "Cloud"])

]

Analyze the employee skill data.

Tasks

1. Find all employees who know Python.
2. Find employees who know both Python and ML.
3. Find employees who know SQL but not Java.
4. Find employees who know either Java or Cloud.
5. Count employees having each skill.
6. Identify the most common skill.
7. Identify employees having at least three skills.
8. Find employees who do not know Python.
9. Sort employees based on the number of skills.
10. Determine which employees are suitable for a Python + ML project.

Expected Output:

Python:

Arun, Charan, Divya, Esha


```
29 repeated_words = [word for word in unique_words if words.count(word) > 1]
30
31 longest_word = max(words, key=len)
32
33 sorted_unique = sorted(unique_words)
34
35 words_with_p = [word for word in unique_words if 'p' in word]
36
37 percentage_unique = (unique_count / total_words) * 100
38
39 print(f"Total Words: {total_words}")
40 print(f"Unique Words: {unique_count}")
41 print()
42 print(f"Frequency of python: {python_freq}")
43 print(f"Frequency of programming: {programming_freq}")
44 print()
45 print("Repeated Words:")
46 for word in repeated_words:
47     print(word)
48 print()
49 print(f"Longest Word:\n{longest_word}")
50 print()
51 print("Words containing 'p':")
52 for word in words_with_p:
53     print(word)
```



Run



Save

Your Code

```
1 import string
2
3 feedback = """
4 Python programming is easy to learn.
5 Python programming is powerful.
6 Learning Python requires practice.
7 Practice makes programming skills better.
8 """
9
10 text = feedback.lower()
11
12 for char in string.punctuation:
13     text = text.replace(char, "")
14
15 words = text.split()
16
17 total_words = len(words)
18
19 unique_words = []
20 for word in words:
21     if word not in unique_words:
22         unique_words.append(word)
23 unique_count = len(unique_words)
24
25 python_freq = words.count("python")
26
27 programming_freq = words.count("programming")
28
29 repeated_words = [word for word in unique_words if words.count(word) >
30
31 longest word = max(words, key=len)
```

Input

stdin input

Output

```
Total Words: 19
Unique Words: 13

Frequency of python: 3
Frequency of programming: 3
```

10 marks

Q8Code Analysis and Debugging
— List + Tuple + String
10 marks**Q9**Word Frequency and Data
Analysis — String + List + Tuple
10 marks**Q10**Integrated Student Data
Analysis — String + List + Tuple
10 marks

10/10 answered

Frequency of python: 3

Frequency of programming: 3

Repeated Words:

python

programming

is

learning

practice

Longest Word:

programming

Words containing 'p':

python

programming

powerful

practice

Your Code

```
1 import string
2
3 feedback = """
4 Python programming is easy to learn.
5 Python programming is powerful.
6 Learning Python requires practice.
7 Practice makes programming skills better.
8 """
9
10 text = feedback.lower()
11
```

Input

stdin input

Output

```
Total Words: 19
Unique Words: 13
Frequency of python: 3
```

QUESTIONS

Q1Student Performance Analysis — List + Tuple
10 marks**Q2**Customer Name Analysis — String + List
10 marks**Q3**Product Inventory Analysis — List + Tuple
10 marks**Q4**Text Analysis — String + List
10 marks**Q5**Employee Skill Analysis — Tuple + List + String
10 marks**Q6**Transaction Analysis — Tuple + List
10 marks**Q7**Password Security Analysis — String + List
10 marks**Q4 Text Analysis — String + List**

10 marks

A university receives the following feedback:

```
feedback = ""
```

Python programming is easy to learn.

Python programming is powerful.

Learning Python requires practice.

Practice makes programming skills better.

```
""
```

Analyze the text.

Tasks

1. Convert the entire text to lowercase.
2. Remove punctuation.
3. Convert the text into a list of words.
4. Count the total number of words.
5. Find the number of unique words.
6. Find the frequency of "python".
7. Find the frequency of "programming".
8. Identify words appearing more than once.
9. Find the longest word.
10. Sort the unique words alphabetically.
11. Find all words containing the letter 'p'.
12. Find the percentage of unique words.

Expected Output:

Total Words: 21

Unique Words: 16

Frequency of python: 3

```
18 total = sum(qty * price for name, qty, price in products)
19 print("Total Inventory Value:", total)
20
21 print("Low Stock:")
22 for name, qty, price in products:
23     if qty < 10:
24         print(name)
25
26 print("Price > 10000:")
27 for name, qty, price in products:
28     if price > 10000:
29         print(name)
30
31 expensive = max(products, key=lambda x: x[2])
32 print("Most Expensive:\n", expensive[0])
33
34 highest = max(products, key=lambda x: x[1] * x[2])
35 print("Highest Inventory Value:\n", highest[0])
36
37 total_qty = sum(qty for name, qty, price in products)
38 print("Total Quantity:", total_qty)
39
40 print("Inventory value more than 50000:")
41 for name, qty, price in products:
42     if qty * price > 50000:
43         print(name)
```

 Run Save

Your Code

```
1 products = [  
2     ("Laptop", 5, 55000),  
3     ("Mouse", 25, 700),  
4     ("Keyboard", 8, 1200),  
5     ("Monitor", 12, 15000),  
6     ("Printer", 4, 12000),  
7     ("Headset", 15, 2500)  
8 ]  
9  
10 values = []  
11  
12 print("Inventory Values:")  
13 for name, qty, price in products:  
14     value = qty * price  
15     values.append((name, value))  
16     print(name, ":", value)  
17  
18 total = sum(qty * price for name, qty, price in products)  
19 print("Total Inventory Value:", total)  
20  
21 print("Low Stock:")  
22 for name, qty, price in products:  
23     if qty < 10:  
24         print(name)  
25  
26 print("Price > 10000:")  
27 for name, qty, price in products:  
28     if price > 10000:  
29         print(name)  
30  
31 expensive = max(products, key=lambda x: x[2])  
32 print("Most Expensive:\n", expensive[0])
```

Input

Output

```
Inventory Values:  
Laptop : 275000  
Mouse : 17500  
Keyboard : 9600  
Monitor : 180000  
Printer : 48000
```


String + List
10 marks

Q8 ✓
Code Analysis and Debugging
— List + Tuple + String
10 marks

Q9 ✓
Word Frequency and Data
Analysis — String + List + Tuple
10 marks

Q10 ✓
Integrated Student Data
Analysis — String + List + Tuple
10 marks

10/10 answered

Expected Output:

Inventory Values:

Laptop : 275000

Mouse : 17500

Keyboard : 9600

Monitor : 180000

Printer : 48000

Headset : 37500

Total Inventory Value: 567600

Low Stock:

Laptop

Keyboard

Printer

Price > 10000:

Laptop

Monitor

Printer

Most Expensive:

Laptop

Highest Inventory Value:

Laptop

Total Quantity: 69

Your Code

```
1 products = [
```

Input

stdin input

CO4_AT2_CASE STUDY[← Back](#)**QUESTIONS****Q1**Student Performance Analysis — List + Tuple
10 marks**Q2**Customer Name Analysis — String + List
10 marks**Q3**Product Inventory Analysis — List + Tuple
10 marks**Q4**Text Analysis — String + List
10 marks**Q5**Employee Skill Analysis — Tuple + List + String
10 marks**Q6**Transaction Analysis — Tuple + List
10 marks**Q7****Q3 Product Inventory Analysis — List + Tuple**

10 marks

A supermarket stores product information as:

```
products = [  
    ("Laptop", 5, 55000),  
    ("Mouse", 25, 700),  
    ("Keyboard", 8, 1200),  
    ("Monitor", 12, 15000),  
    ("Printer", 4, 12000),  
    ("Headset", 15, 2500)  
]
```

Each tuple contains:

(Product Name, Quantity, Price)

Analyze the inventory.

Tasks

1. Calculate the inventory value of every product.
2. Calculate the total inventory value.
3. Identify products whose quantity is less than 10.
4. Identify products whose price is greater than ₹10,000.
5. Find the most expensive product.
6. Find the product having the highest inventory value.
7. Sort products based on price.
8. Sort products based on quantity.
9. Calculate the total quantity of all products.
10. Identify products contributing more than ₹50,000 to inventory value.

```
25 print("Number of Unique Names:", len(unique))
26
27 multi = []
28 for name in unique:
29     if len(name.split()) > 1:
30         multi.append(name)
31 print("Names with Multiple Words:\n", multi)
32
33 longest = max(unique, key=len)
34 print("Longest Name:", longest)
35
36 start = []
37 for name in unique:
38     if name.startswith("a") or name.startswith("d"):
39         start.append(name)
40 print("Starts with A or D:", start)
41
42 print("Character count:")
43 for name in normalized:
44     print(name, ":", len(name))
45
46 sorted_names = sorted(unique)
47 print("Sorted names:", sorted_names)
48
49 most = max(unique, key=normalized.count)
50 print("Most frequent:", most)
```

 Run Save

Your Code

```
1 names = [  
2     " Arun Kumar ",  
3     "ARUN KUMAR",  
4     " arun kumar",  
5     "Bala Krishnan",  
6     "BALA KRISHNAN ",  
7     "Charan",  
8     " charan ",  
9     "DIVYA DEVI",  
10    "Divya Devi"  
11 ]  
12  
13 normalized = [name.strip().lower() for name in names]  
14 print("Normalized:\n", normalized)  
15  
16 duplicates = []  
17 for name in normalized:  
18     if normalized.count(name) > 1 and name not in duplicates:  
19         duplicates.append(name)  
20 print("Duplicate Customers:\n", duplicates)  
21  
22 unique = list(set(normalized))  
23 unique.sort()  
24 print("Unique Customers:\n", unique)  
25 print("Number of Unique Customers:", len(unique))  
26  
27 multi = []  
28 for name in unique:  
29     if len(name.split()) > 1:  
30         multi.append(name)
```

Input

Output

```
Normalized:  
['arun kumar', 'arun kumar',  
'arun kumar', 'bala krishnan',  
'bala krishnan', 'charan',  
'charan', 'divya devi', 'divya  
devi']
```

10 marks

Q8 Code Analysis and Debugging
— List + Tuple + String
10 marks**Q9** Word Frequency and Data
Analysis — String + List + Tuple
10 marks**Q10** Integrated Student Data
Analysis — String + List + Tuple
10 marks

10/10 answered

Expected Output:

Normalized:

```
['arun kumar', 'arun kumar', 'arun kumar',  
'bala krishnan', 'bala krishnan',  
'charan', 'charan',  
'divya devi', 'divya devi']
```

Unique Customers:

```
['arun kumar', 'bala krishnan', 'charan', 'divya devi']
```

Number of Unique Customers: 4

Duplicate Customers:

```
arun kumar  
bala krishnan  
charan  
divya devi
```

Names with Multiple Words:

```
arun kumar  
bala krishnan  
divya devi
```

Longest Name:

```
bala krishnan
```

Starts with A or D:

```
arun kumar  
divya devi
```

Your Code

Input

QUESTIONS

Q1Student Performance Analysis
— List + Tuple
10 marks**Q2**Customer Name Analysis —
String + List
10 marks**Q3**Product Inventory Analysis —
List + Tuple
10 marks**Q4**Text Analysis — String + List
10 marks**Q5**Employee Skill Analysis — Tuple
+ List + String
10 marks**Q6**Transaction Analysis — Tuple +
List
10 marks**Q7**Password Security Analysis —
String + List
10 marks**Q2 Customer Name Analysis — String + List**

10 marks

An e-commerce company receives customer names in inconsistent formats:

```
names = [
```

```
    "Arun Kumar ",
```

```
    "ARUN KUMAR",
```

```
    " arun kumar",
```

```
    "Bala Krishnan",
```

```
    "BALA KRISHNAN ",
```

```
    "Charan",
```

```
    " charan ",
```

```
    "DIVYA DEVI",
```

```
    "Divya Devi"
```

```
]
```

Analyze the customer data.

Tasks

1. Remove unnecessary leading and trailing spaces.
2. Convert all names to lowercase.
3. Identify duplicate customers.
4. Count the number of unique customers.
5. Find customers whose names contain more than one word.
6. Find the longest customer name.
7. Find customers whose names start with 'a' or 'd'.
8. Count the number of characters in each normalized name.
9. Sort the unique names alphabetically.
10. Determine which customer name occurs most frequently.

Expected Output:


```
33
34 print("\nAbove 80 in at least 3 subjects:")
35 for name, marks in students:
36     count = 0
37     for mark in marks:
38         if mark > 80:
39             count += 1
40     if count >= 3:
41         print(name)
42
43 print("\nHighest mark in each subject:")
44 for i in range(4):
45     print(max(marks[i] for name, marks in students))
46
47 print("\nBelow 50 in any subject:")
48 for name, marks in students:
49     if any(mark < 50 for mark in marks):
50         print(name)
51
52 results.sort(key=lambda x: x[2], reverse=True)
53
54 print("\nStudents sorted by average:")
55 for x in results:
56     print(x[0], x[2])
57
58 print("\nTop two students:")
59 print(results[0][0])
60 print(results[1][0])
61
```



Run



Save

Your Code

```
1 students = [  
2     ("Arun", [78, 85, 92, 67]),  
3     ("Bala", [45, 67, 55, 49]),  
4     ("Charan", [90, 92, 88, 95]),  
5     ("Divya", [65, 72, 70, 68]),  
6     ("Esha", [88, 76, 91, 84])  
7 ]  
8  
9 results = []  
10  
11 print("Student Totals and Averages:")  
12  
13 for name, marks in students:  
14     total = sum(marks)  
15     avg = total / len(marks)  
16  
17     results.append((name, total, avg))  
18  
19     print(name, ":", total, ",", avg)  
20  
21 highest = max(results, key=lambda x: x[1])  
22 print("\nHighest Total:", highest[0])  
23  
24 lowest = min(results, key=lambda x: x[2])  
25 print("Lowest Average:", lowest[0])  
26  
27 class_avg = sum(x[2] for x in results) / len(results)  
28  
29 print("\nAbove Class Average:")  
30 for name, total, avg in results:  
31     if avg > class_avg:  
32         print(name)
```

Input

stdin input

Output



10 marks

Q8

Code Analysis and Debugging

— List + Tuple + String

10 marks

Q9

Word Frequency and Data

Analysis — String + List + Tuple

10 marks

Q10

Integrated Student Data

Analysis — String + List + Tuple

10 marks

10/10 answered

Divya : 275

Esha : 339

Student Averages:

Arun : 80.50

Bala : 54.00

Charan : 91.25

Divya : 68.75

Esha : 84.75

Highest Total: Charan

Lowest Average: Bala

Above Class Average:

Arun, Charan, Esha

Students with 3 or more marks above 80:

Charan

Subject-wise Highest:

Subject 1: 90

Subject 2: 92

Subject 3: 92

Subject 4: 95

Students with a mark below 50:

Bala

Top 2:

Charan

Esha

QUESTIONS

Q1Student Performance Analysis — List + Tuple
10 marks**Q2**Customer Name Analysis — String + List
10 marks**Q3**Product Inventory Analysis — List + Tuple
10 marks**Q4**Text Analysis — String + List
10 marks**Q5**Employee Skill Analysis — Tuple + List + String
10 marks**Q6**Transaction Analysis — Tuple + List
10 marks**Q7**Password Security Analysis — String + List
10 marks**Q1 Student Performance Analysis — List + Tuple**

10 marks

A college stores the following student records:

```
students = [  
    ("Arun", [78, 85, 92, 67]),  
    ("Bala", [45, 67, 55, 49]),  
    ("Charan", [90, 92, 88, 95]),  
    ("Divya", [65, 72, 70, 68]),  
    ("Esha", [88, 76, 91, 84])  
]
```

Analyze the student data and perform the following operations:

1. Calculate the total and average marks of every student.
2. Identify the student with the highest total.
3. Identify the student with the lowest average.
4. Find students whose average is greater than the class average.
5. Find students who scored above 80 in at least three subjects.
6. Find the highest mark in each subject.
7. Identify students who scored below 50 in any subject.
8. Sort the students based on their average marks in descending order.
9. Identify the top two students.
10. Explain which parts of the structure are mutable and which are immutable.

Expected Output:

Student Totals:

Arun : 322

Bala : 216

Charan : 365

Divya : 275